

GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY

DEPARTMENT OF MECHANICAL ENGINEERING

MECHATRONICS SYSTEM DESIGN

AUTONOMOUS DELIVERY ROBOT CHALLENGE

GROUP NO	09
TEAM MEMBERS	M.K.T. GUNAWARDENA [0075] G.P. NAWARATHNA [0189] M.A.N. NAVEENA [0061]
MODULE CODE	MC 3113
SUBMISSION DATE	24/06/2025

TABLE OF CONTENTS

INTRODUCTION 3
ROBOT DESIGN AND CONSTRUCTION 4
System design 4
Component description 4
Innovative Pick-and-Place method. 5
Using HMI with Innovative Pick-and-Place Method. 6
Robot's color / shape memory ability. 6
Precise movement. 6
Design choices. 6
Wiring and layout 6
Software Development for the line following. 7
Conceptual design 8
Robot arm 9
Insert the color sensor. 10
Communication Between Raspberry Pi and ESP32 (First Stage) 10
Color detection using the RASPBERRY PI 11
Object shape detection using ESP32-CAM 11
Final wiring diagram 13
Final budget for project. 14
SOFTWARE AND HMI DEVELOPMENT. 15
HMI development using ESP32 and BLYN. 15
Software development 16
SIMULATION RESULTS AND MATHEMATICAL MODELS 19
Calculations 19
ROBOT PATH SIMULINK. 21
ANSYS Simulation of Robot Arm 25
CHALLENGES ENCOUNTERED AND SOLUTIONS IMPLEMENTED. 28
REFERENCES 33

INTRODUCTION

This project is about building an autonomous delivery robot. The robot is made to move on its own, follow a black line, find objects by their color or shape, and carry them to a given place without help from a human.

The robot uses 11 IR (3 single IR and 8 IR array) sensors to follow the black line. It also uses color sensors to detect objects based on color. A small robot arm with 2 servo motors is used to pick up and place objects. The robot can carry objects up to 110 g.

The brain of the robot is the ESP32 and Arduino-nano microcontrollers. It helps the robot to think and make decisions quickly. The robot can also handle changes like different light levels or small blocks on the path. A Human-Machine Interface (HMI) is used for controlling and decision the color.

Before building the real robot, we tested it using a MATLAB simulation. This helped us fix problems before making the real one. We created the mechanical design using SOLIDWORKS 2024, and we kept the robot small to match the size rules (15 cm long and 12 cm wide). The robot is powered by a 7.4V LiPo battery, and it moves using N20 gear motors, which give good power and smooth movement. During the project, we solved problems like keeping the power stable, turning accurately, and detecting colors under different lights. This project shows how mechatronics can solve real problems, and it also shows our teamwork and creativity in building smart systems.

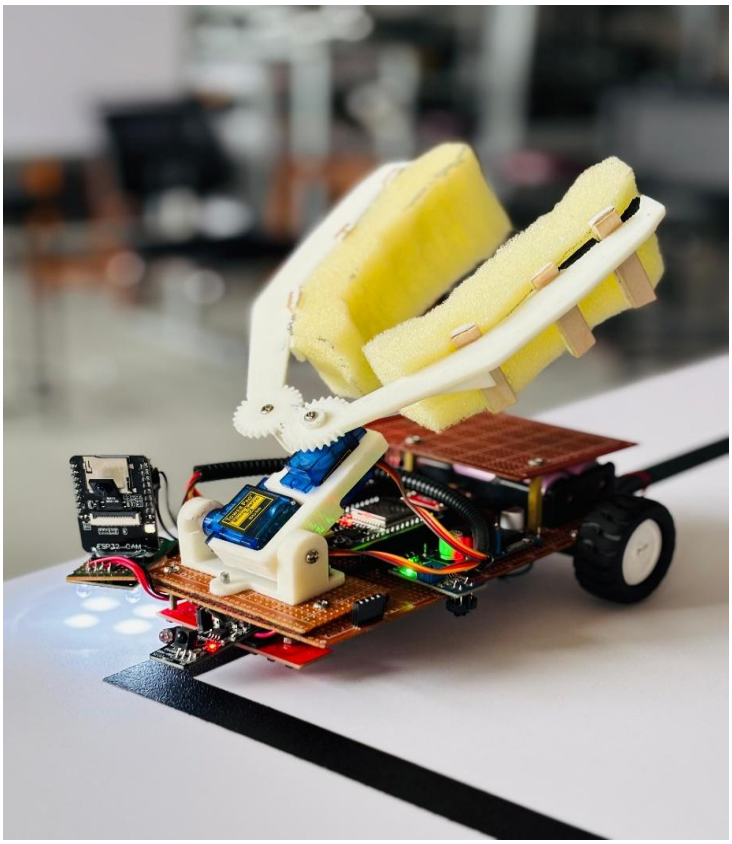
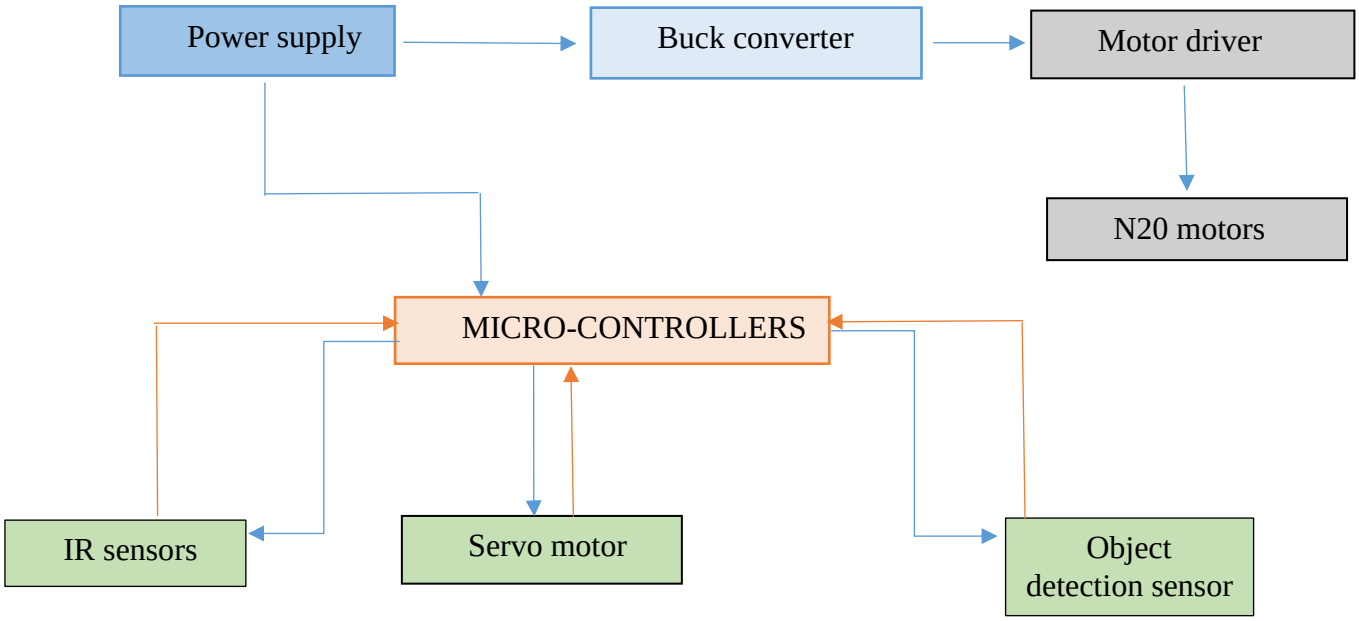


Figure 1- Final output

ROBOT DESIGN AND CONSTRUCTION

- System design



- Component description

Component	Function	Specifications
Esp32	For line following and sensor input.	Flash Memory: 4 MB ADC:12-bit, up to 18 channels Power Supply: 5V
Arduino nano	Controlling robot arm, color identification.	Operating voltage :5V Flash Memory: 32 KB
8 array IR sensor / single 3 IR sensors	Detecting the line, object detection.	Operating Voltage: 3.3V Detection Range: 1cm
N20 gear motors	For motion.	Operating Voltage: 6V Speed: 200RPM Torque: 1kg.cm

Motor driver	Controlling of the motors.	Operating Voltage: 7.2V Current Capacity: 2A per channel PWM Support: Yes
Battery	Main power source.	(Lithium Polymer - LiPo) Capacity: 2000mAh Single cell: 3.7V Charging Voltage: 4.2V per cell
Color sensor (TCS230)	Identify the color.	4 types of photodiodes (Red, Green, Blue, and Clear)
Servo motors	Controlling arm movements in object handling.	SG90 Micro Servo (Light Duty) Operating Voltage: 5V Torque: 500g.cm @ 5V Rotation Angle: 0° – 180°
ESP32-CAM	Image Capture, Object identify.	Camera-OV2640(2Megapixels) RAM: 520 KB SRAM Operating Voltage: 3.3V
ON/OFF switch	Power Control.	Voltage Rating**:** 5V – 12V DC Current Rating**:** Up to 2A

- Innovative Pick-and-Place method.

In this project, we needed to pick and place two objects one by one. However, our robot was able to pick both objects at the same time and place them one after another. This is a new, efficient method we developed.

- ✓ First, the robot picks the second object and stores it safely on its back. Then, it picks the first object with the robotic arm and moves to the delivery place.
- ✓ At the place, the robot places the first object first, and then it places the second object from its back.

This way, the robot can complete the task faster and more efficiently than picking and placing objects one by one.

- Using HMI with Innovative Pick-and-Place Method.

In this method, we use an HMI (Human-Machine Interface) to select the two needed objects. The user touches the HMI screen to choose the first and second objects. The robot then picks the second selected object first and stores it on its back. After that, it picks the first selected object using the robot arm and moves to the placing area. At the place, the robot puts the first object first, and then places the second one. This smart method saves time and follows the user's exact selection made on the HMI. To move smoothly and accurately and pick relevant object, the robot uses PID control while following the line path and object detection.

- Robot's color / shape memory ability.

The robot can remember the color and shape of the second object while it picks the first one. When the robot picks the first object, it stores the color/shape information of the second object in its memory. After storing the first object, the robot uses this stored color/shape information to go directly to the second object's location without searching. This ability helps the robot work faster and more smartly by planning its path ahead.

- Precise movement.

The precise movement of the robot is ensured by using two N20 motors to drive its front wheels. N20 motors respond smoothly to PWM signals making it easier to adjust the speed and direction of the robot precisely. Since the precise directional control of the caster wheel is harder to achieve, we have integrated the caster wheel behind the front wheels. In that way, the caster wheel can rotate in accordance with the front wheels, ensuring the stability of the chassis.

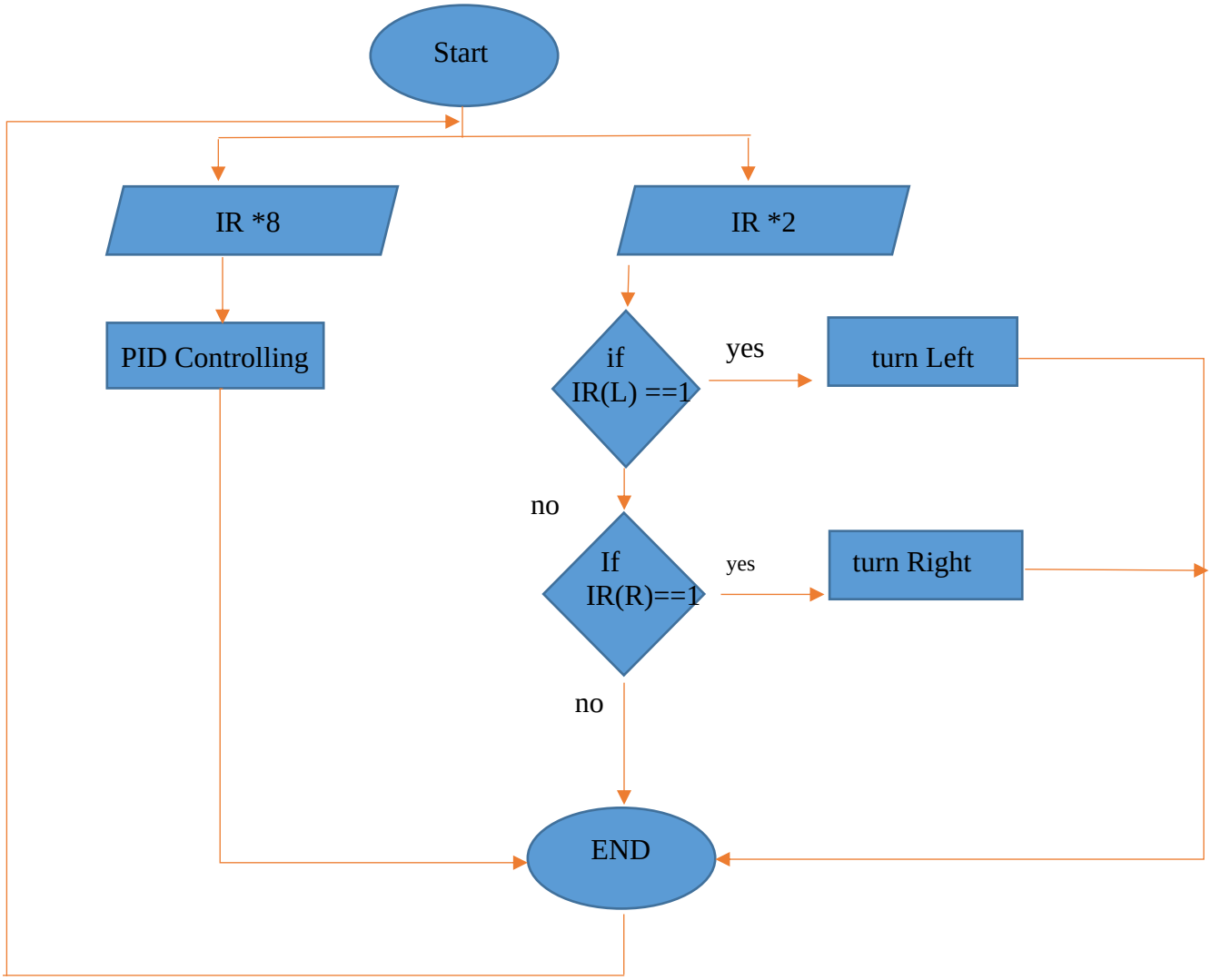
- Design choices.

To keep the robot within the specified dimensions, we have used 2 chassis plates, one on top of the other to enlarge the space for component integration. When integrating chassis plates with the N20 motors and wheel. For the distribution of weight, the battery is placed at the back of the robot, and other components at the front. The placement of other components is arranged according to the ease of wiring and handling. 11 IR sensors are placed at specified distances and closer to the ground so that the detection of the black line is accurate. The arm used for the box replacement is selected as per the design simplicity and accuracy.

- Wiring and layout.

We have started the project using Raspberry-Pi and ESP32. During the process, Raspberry-Pi board was down due to an emergency case. As a solution we decided to only run this from ESP32 and Arduino-nano. Power management is done by giving a power supply of 7.2V directly to the buck converter. To give power supply to the sensors and other actuators voltage is reduced to 6V by using a buck converter.

- Software Development for the line following.



- Conceptual design

  ✓ Robot computerized design (Detailed CAD model)

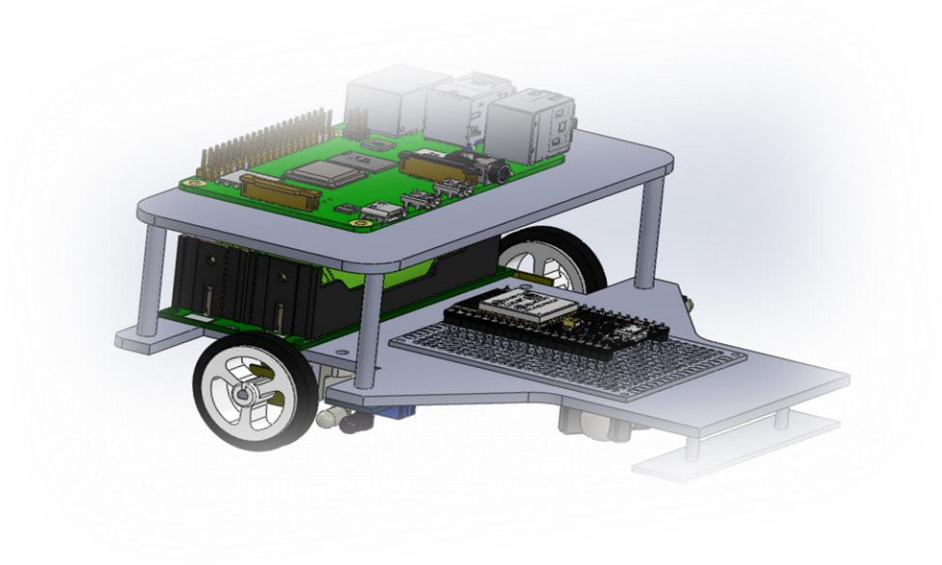


Figure 2- Detailed CAD model.

We made a full 3D model of our delivery robot using SOLIDWORKS. This model included all the parts mechanical, electrical, and sensors. Making the model helped us check the design on the computer, improve how it works, and make building it easier. The design was ready for the competition and was made to be accurate, flexible, and low-cost.

But when we were building the robot, the Raspberry Pi board stopped working. So, we changed it and used an Arduino and an ESP32 instead. Even though we changed the real parts in the robot, we did not change the computer design. We used the same CAD model. This shows that our design was strong and flexible, and we were able to solve problems and keep going without starting over.

  ✓ Robot final design according to the conceptual design

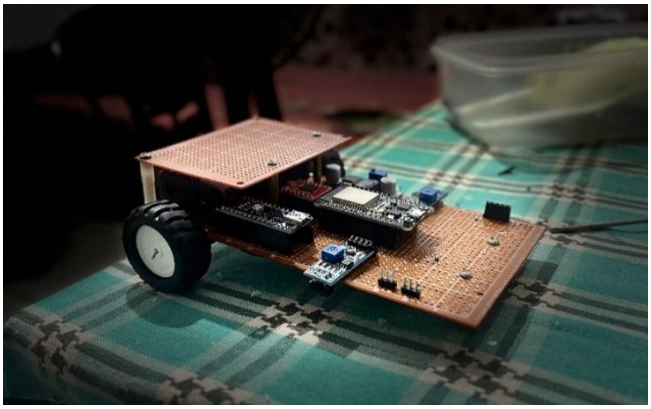


Figure 3-Final design for the chassis plate

- Robot arm

  ✓ Detailed SOLIDWORKS model

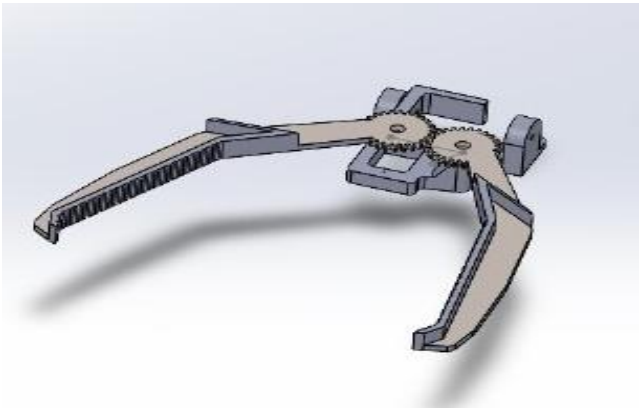


Figure 4-Robot arm using SOLIDWORKS.

We have developed a special robotic arm for our autonomous delivery robot. We designed the arm using SOLIDWORKS and made it use a 3D printer. First, we created a careful 3D model to make sure the arm is strong and works well. Then, we printed it using high-quality PLA material, which is strong but also light this is important for our robot to work properly.

The robotic arm can move 40mm sideways (horizontally) and 160 degrees up and down (vertically). Inside the arm, we added a sponge, which helps the arm hold and pick up a 3cm × 3cm cube accurately. This design helps the robot handle small items safely and correctly.

  ✓ Final design of the arm

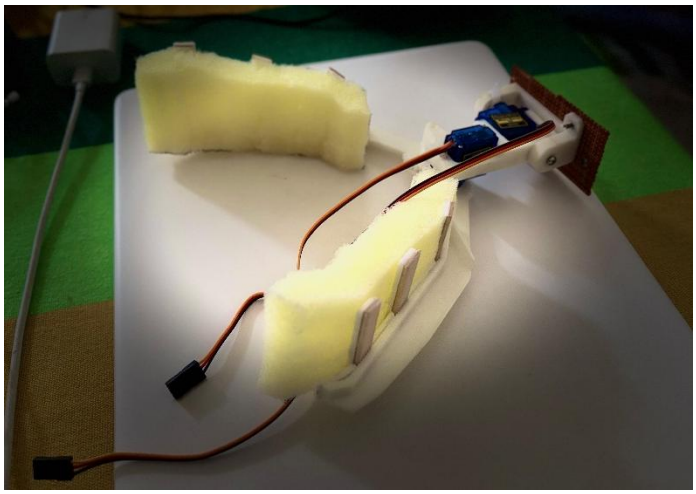


Figure 5- 3D printed robot arm

- Insert the color sensor.

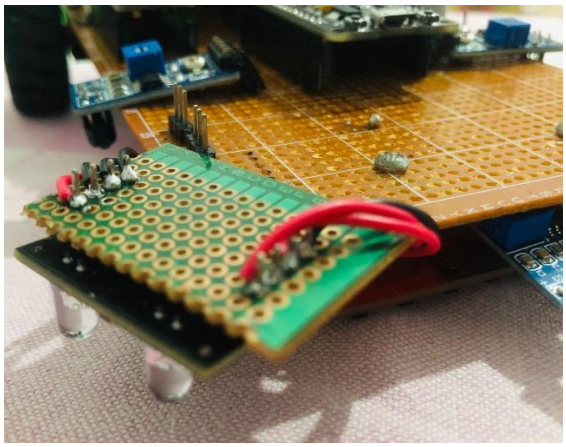


Figure 6-Color sensor

Finally, we added a color sensor to the robot. So, we placed it on the right-hand side at the front of the chassis. This position still allows the sensor to detect colors in front of the robot. We made sure it was fixed properly and worked well with the system.

- Communication Between Raspberry Pi and ESP32 (First Stage)

In the first stage of the project, we used a Raspberry Pi board as the main controller. The Raspberry Pi communicated with the ESP32 using serial communication (UART**)**. The Raspberry Pi sent commands to the ESP32 when an object was detected. The ESP32 then handled sensor readings, motor control, and actuator movements based on these commands. This setup allowed smooth coordination between the two boards and helped the robot perform its tasks correctly. However, due to technical issues with the Raspberry Pi, we later changed the system by fully shifting control to the ESP32 and Arduino Nano.

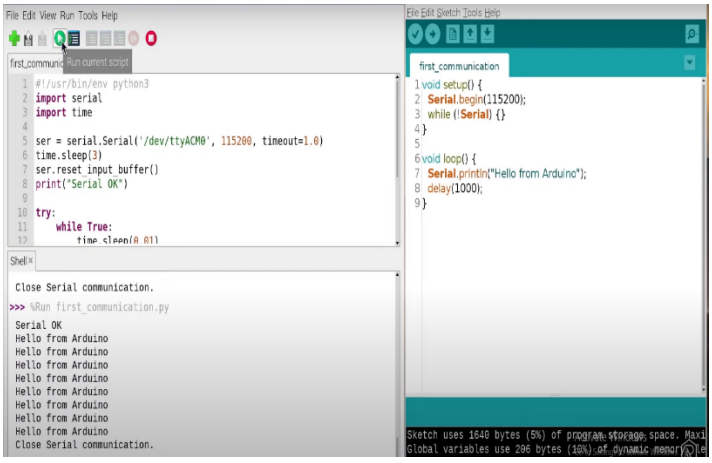


Figure 7-Communication between the RASPBERRY PI and ESP32

- Color detection using the RASPBERRY PI

Implemented color detection on the robot using Raspberry Pi for accurate object identification and task execution.

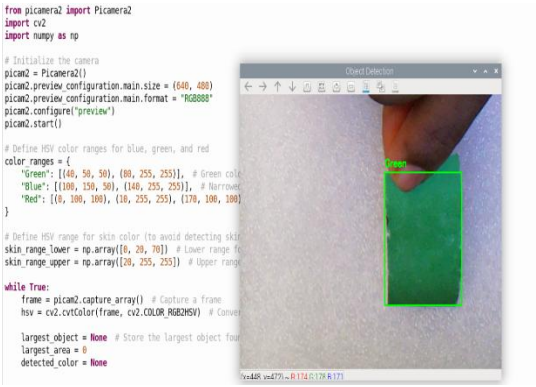


Figure 8-Color detection for green

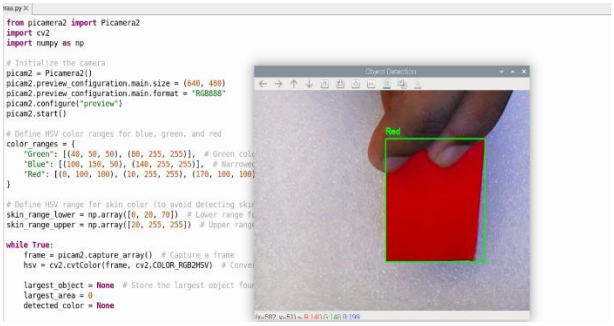


Figure 9-Color detection for red

In the first stage of the project, we used a Raspberry Pi to perform color detection for accurate object identification. The Raspberry Pi used a camera module to capture images and detect the color of objects.

- Object shape detection using ESP32-CAM

In this project, we used the ESP32-CAM module to perform object shape detection**.** The camera captured real-time images, and the program processed these images to identify three basic shapes of cube**,** sphere**,** and cylinder. By analyzing the outline and features of the object in front of the camera, the system could classify the shape correctly. This feature helped the robot decide which object to pick based on the selected shape.

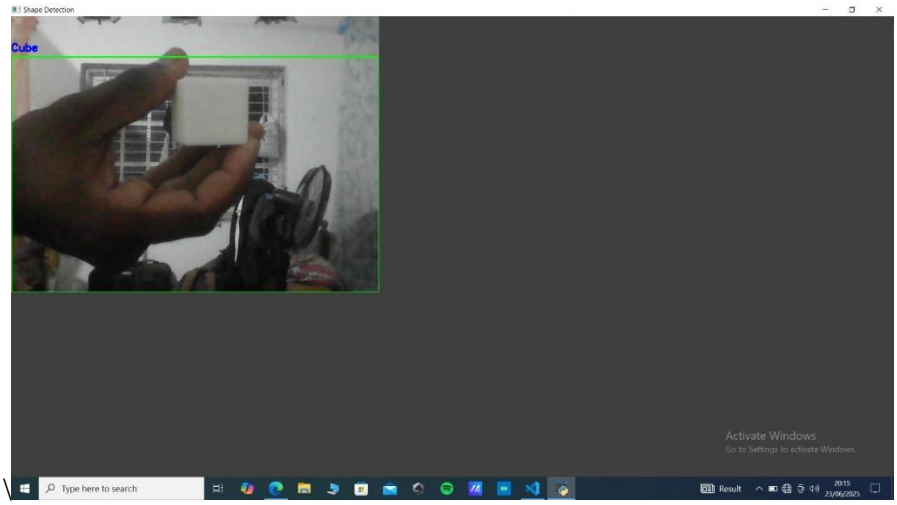


Figure 10-Cube detection

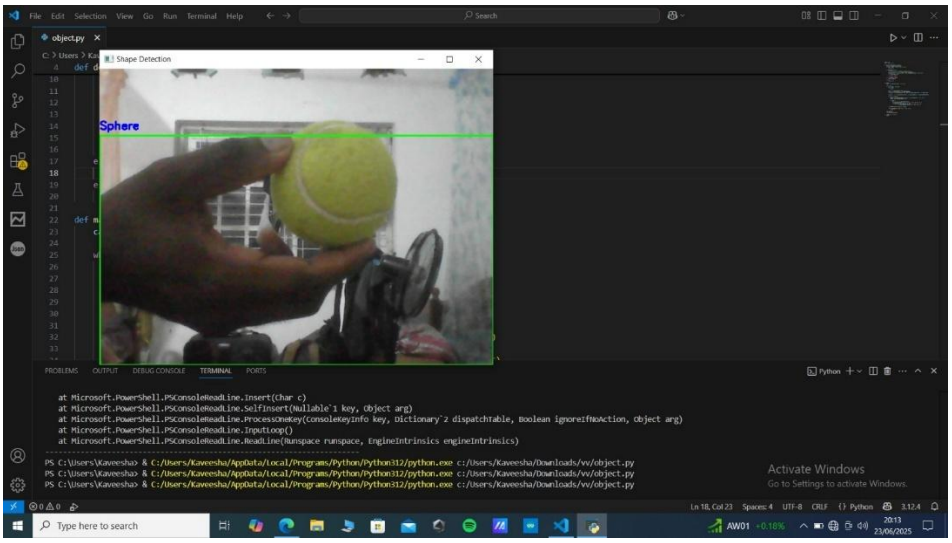


Figure 11-Sphere detection

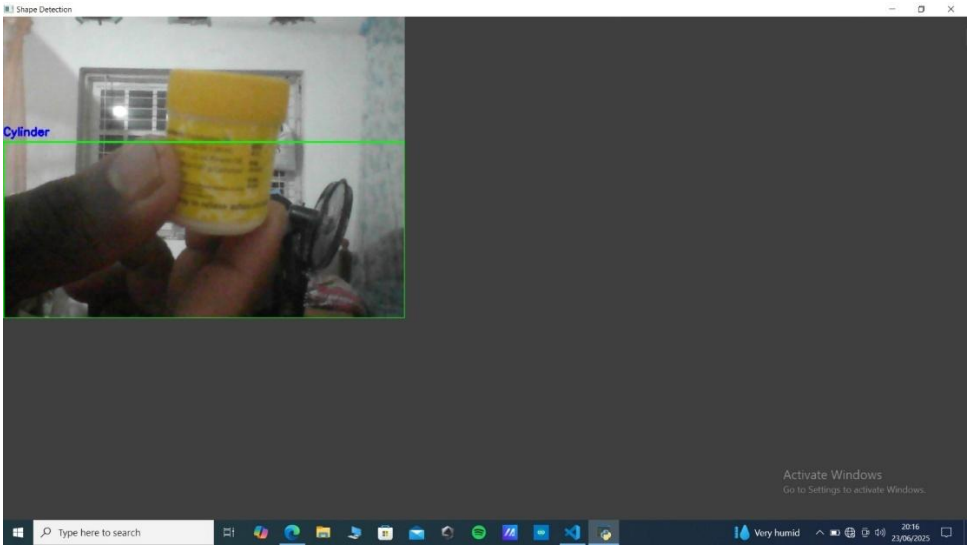


Figure 12-Cylinder detection

- Final wiring diagram

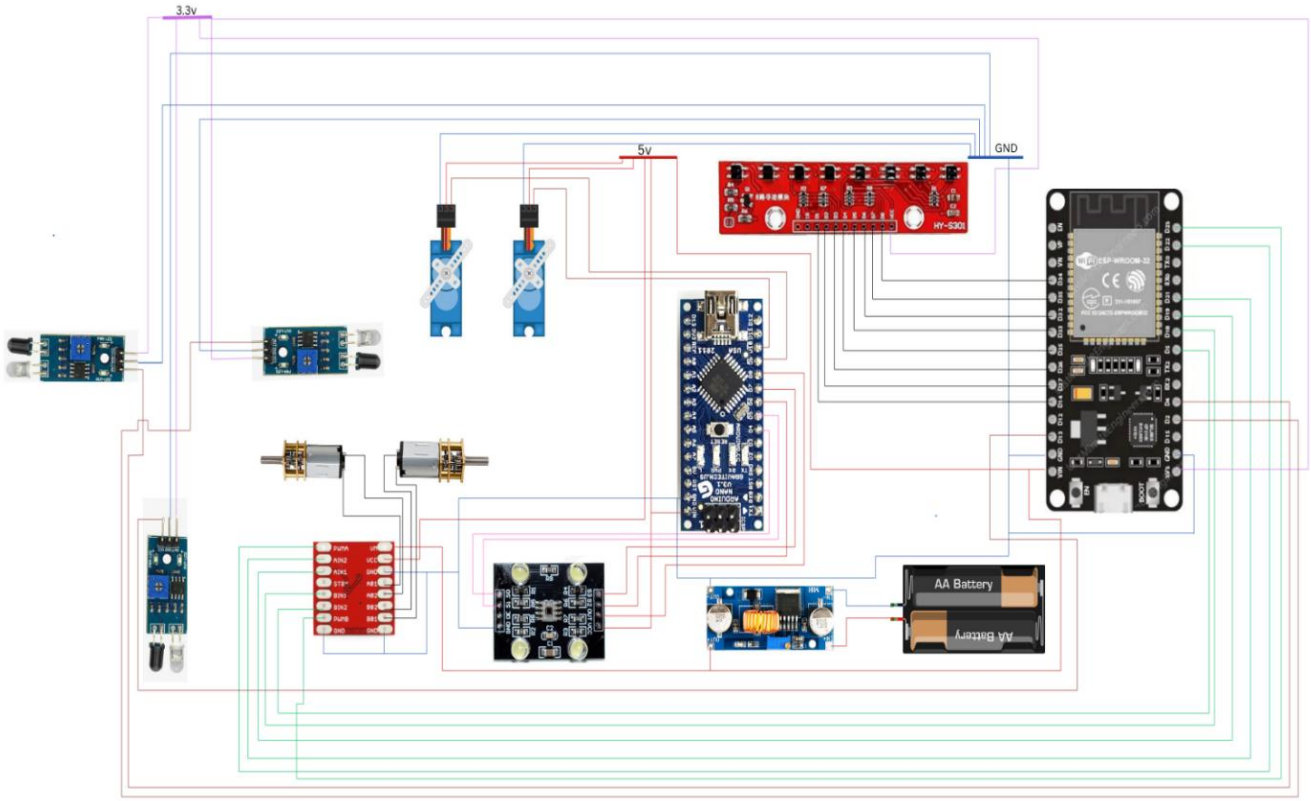


Figure 13 - wiring diagram

- Final budget for project.

Component	Quantity	Cost per one [Rs]	Total cost [Rs]
ESP32	1	1500	1500
Arduino nano board	1	1700	1700
QTR	1	800	800
IR sensor	3	190	570
Buck converter	1	300	300
Motor driver	1	350	350
N20 motor with bracket	2	1100	2200
Arm	1	1520	1520
Servo motors	2	480	960
Color sensor	1	760	760
Dot board (chassi)	1	80	80
wheels	2	145	290
Esp32 cam module	1	1800	1800
Battery with case	2	395	790
Wires	5m	50	250
switch	2	50	100
TOTAL BUDGET			**13970**

Damaged components

Component	Quantity	Cost per one [Rs]	Total cost [Rs]
Raspberry Pi board	1	-	-
ESP32	1	1500	1500
IR sensors	3	190	570
Servo motors	3	480	1440
TOTAL BUDGET			**3510**

SOFTWARE AND HMI DEVELOPMENT.

- HMI development using ESP32 and BLYN.

This project focuses on designing and controlling it in real-time through a Human-Machine Interface (HMI). The HMI was developed using Blynk and is connected to the robot using an ESP32 microcontroller over Wi-Fi. The main goal is to provide wireless control and task selection such as obstacle detection, and color recognition.

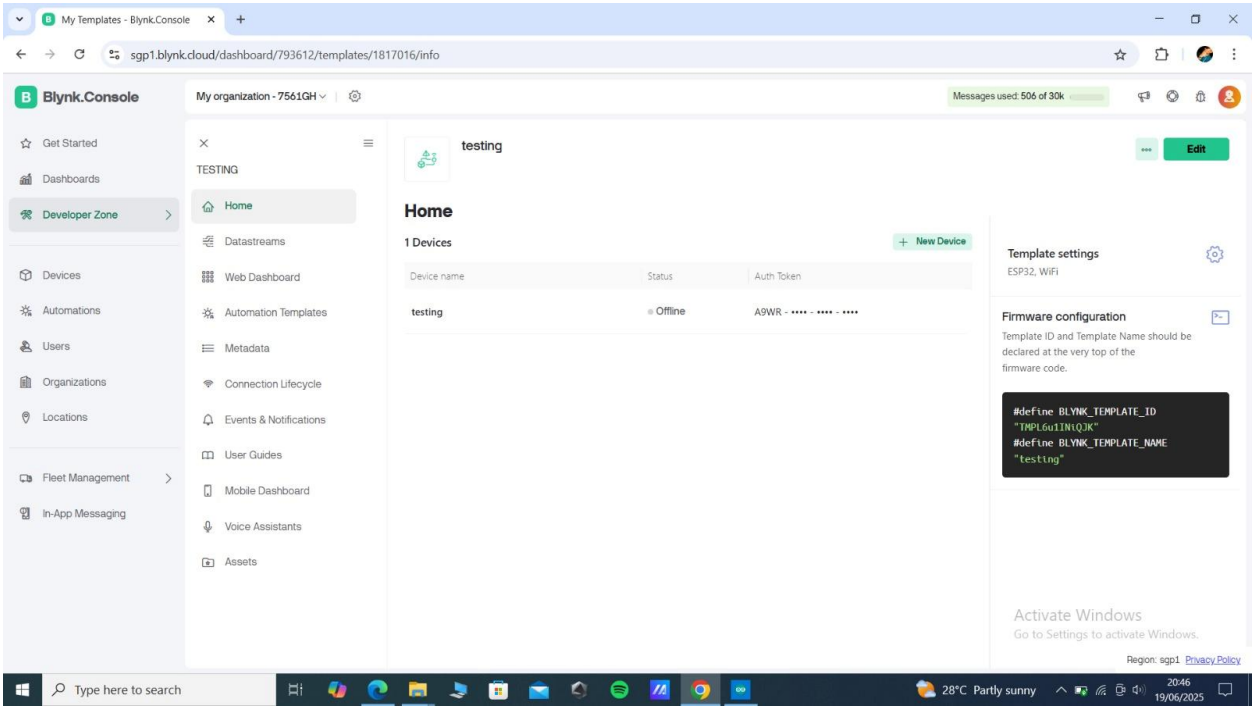


Figure 14-BLYN's initial interface

In this project, we connect the ESP32 microcontroller to a Human-Machine Interface using the Blynk app on a computer. First, we open the Blynk app and make a new project. We choose ESP32 as the device and Wi-Fi as the way to connect. The app sends a special code called Auth Token to our email. We put this code, our Wi-Fi name, and password into the ESP32 program. Then, we write and upload the program to the ESP32 using the Arduino IDE. When we turn on the ESP32, it connects to the Wi-Fi and talks to the Blynk server using the Auth Token. Now, the Blynk app on the laptop works as the HMI. We can use buttons and sliders on the app to control and watch the ESP32 from far away using the internet.

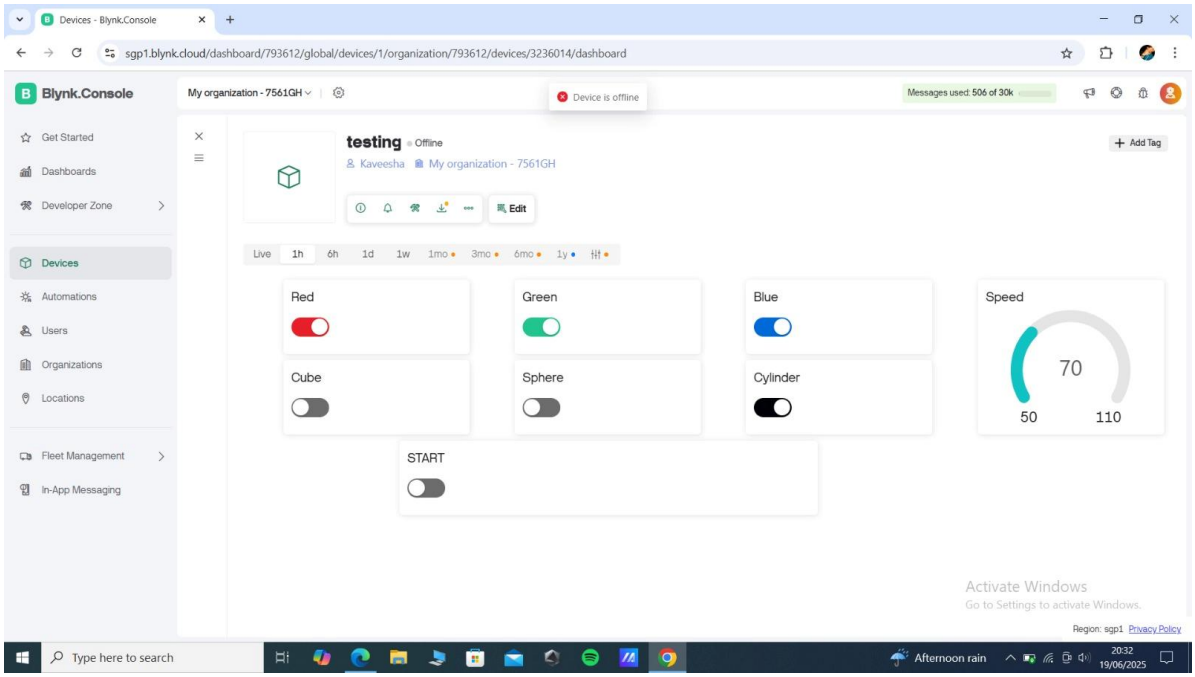


Figure 15-HMI for the robot

On the HMI display, the user can choose the color and shape of the object using switch buttons**.** The robot will only start its operation after both options are selected**.** This ensures the robot follows the correct instructions for the pick-and-place task.

Additionally, the HMI also shows the speed of the robot in real-time. This helps the user know when the speed is low, making the system more reliable and easier to use.

- Software development

The code for our robot was developed using a flowchart-based method, which helped us organize and control each step of the robot's actions clearly. The robot follows a super method that manages line following, object picking and placing, and color memorization in the correct order. Every movement, whether at a straight line, a junction, or a turning point is handled based on this logical flow.

The program checks the sensor inputs and follows the path according to the conditions shown in the flowchart. For example, after detecting the selected color, the robot stores that information in memory and continues with the next action without repeating steps. This method made the robot smart, efficient, and able to complete complex tasks without confusion. The full flowchart is attached in this documentation to show how each part of the robot's logic works step by step.

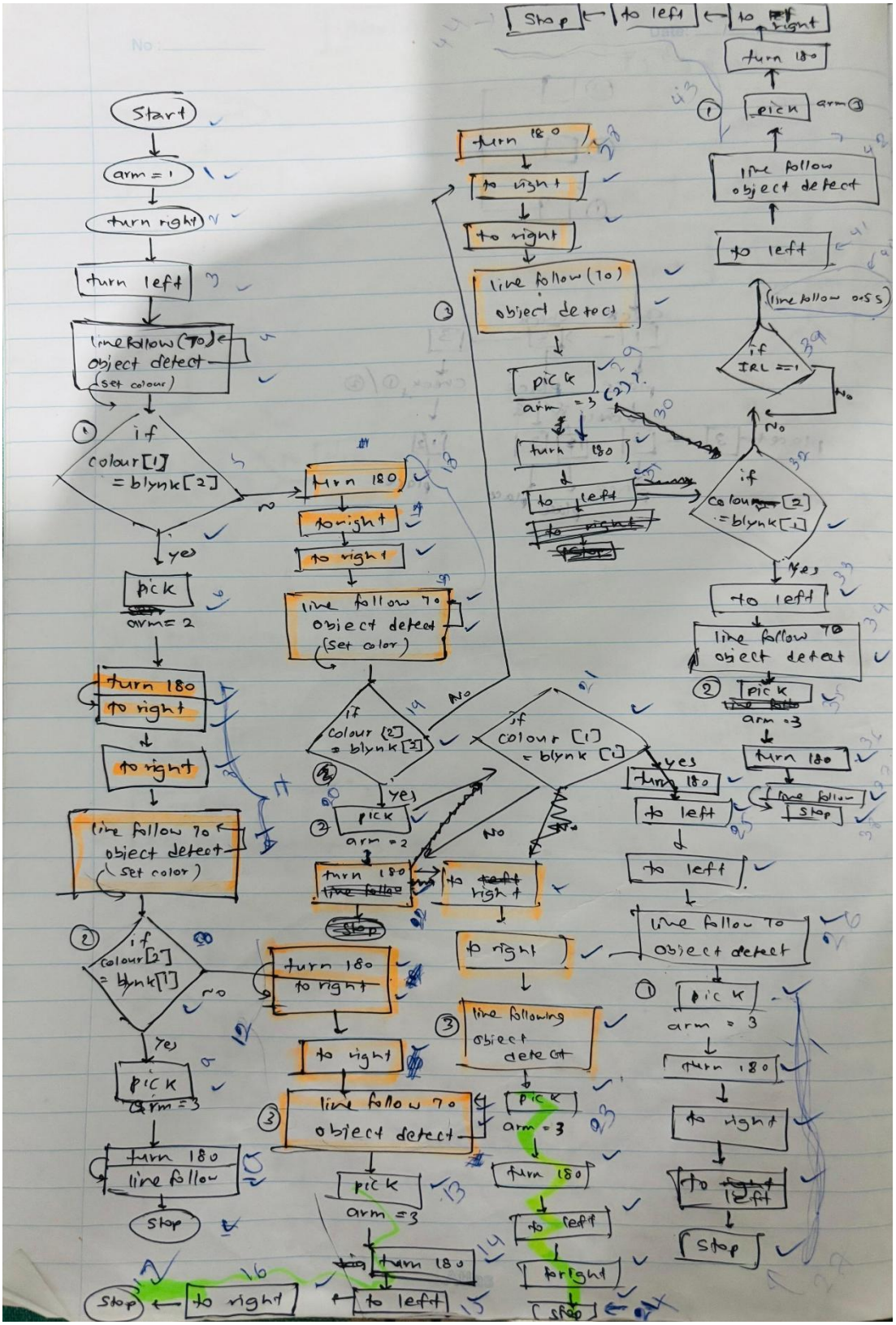


Figure 16-Flow chart page 1

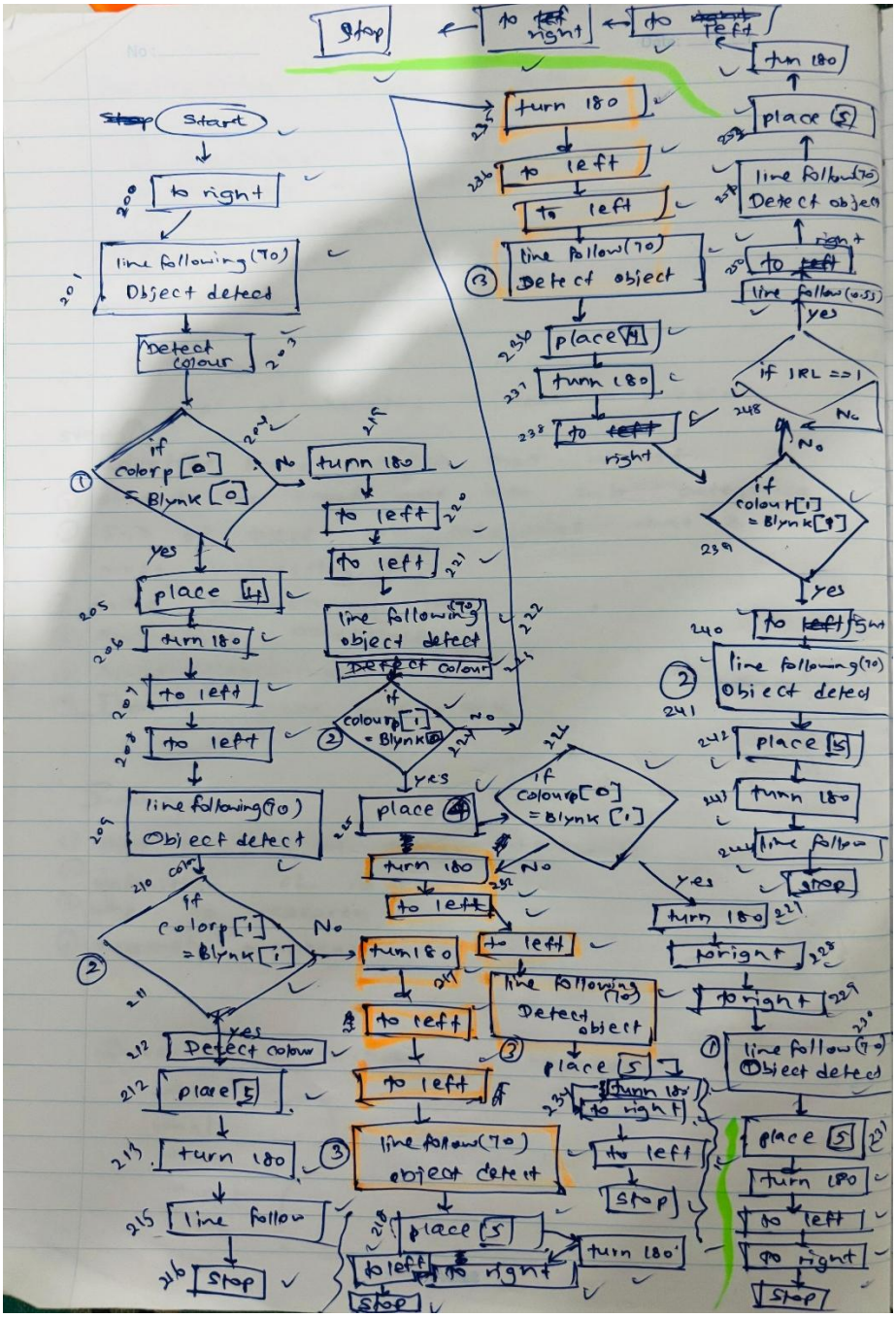


Figure 17-flow chart page 2

Using the flowchart method was very helpful for our software development. It allowed us to organize all the steps of the robot's work clearly from the start to the end point. Each action like line following, object detection, picking, placing, and color memory was shown step by step. This made it easy to understand the full process and how the robot should behave in different situations. The flowchart also helped us identify the function of each part of the program and made it easier to find and fix any errors during testing.

SIMULATION RESULTS AND MATHEMATICAL MODELS

- Calculations

  ✓ Battery calculations

  N20 motors = 480mA
  Servo motors = 400mA
  Other components = 250+50+60+30+10+80 = 480mA

  Total = 1.36A

  Battery = runtime * total current draw
  = 1 Hour * 1.36A
  = 1360mAh

  ❖ So 1800mAh is enough.

  ✓ Motor

  Wheel radius = 21.5mm
  C = 2 (Pie) r
  = 0.135mm

  V = CN/60
  = 0.135*200/60
  = 0.45
  Total mass = 450g

  Torque (T) = $0.45Kg*9.81ms^{-1}*21.5*10^{-3}m$
  = 47.4Nm

  ❖ N20 motors give 100Nm. So N20 motors 200RPM, 6V is enough to move the robot.

  ✓ Gripper arm
  Length of gripper arm (L)= 80mm
  Mass of gripper = 0.15Kg
  Mass of cube = 0.1g
  Coefficient of friction(Ч) = 0.5

F=mg (for cube)
= 0.1Kg* 9.81ms^{-2}
= 0.981N

$F_{(finger)}$ = F/ 2Ч
= 0.981/ (2*0.5)
= 0.981N

T = $F_{(finger)}$ * L
= 0.981 * 0.08
= 0.07848Nm

Torque from gripper weight
$T_{(f)}$= $W_{(f)}$ * (L/2)
= 0.05*9.81*0.08/2
= 0.01962Nm

$T_{(total)}$= T + $T_{(f)}$
= 0.07848+0.01962
= 0.0981Nm

SG90 Torque = 0.176Nm

[0.0981 < 0.176]

- SG90 is sufficient to lift 100g box with length of 80mmarm.

- ROBOT PATH SIMULINK.

This robot follows a black line on a white surface using sensors and a control system. In Simulink, we can simulate this behavior without needing to build the actual robot. The model uses virtual sensors to detect the line, a PD (Proportional-Derivative) controller to figure out how much the robot needs to turn, and motor blocks to control the wheel speeds. This makes it easy to test and improve how the robot follows the path, helping us understand the basics of feedback control and robotics in a safe, flexible way.

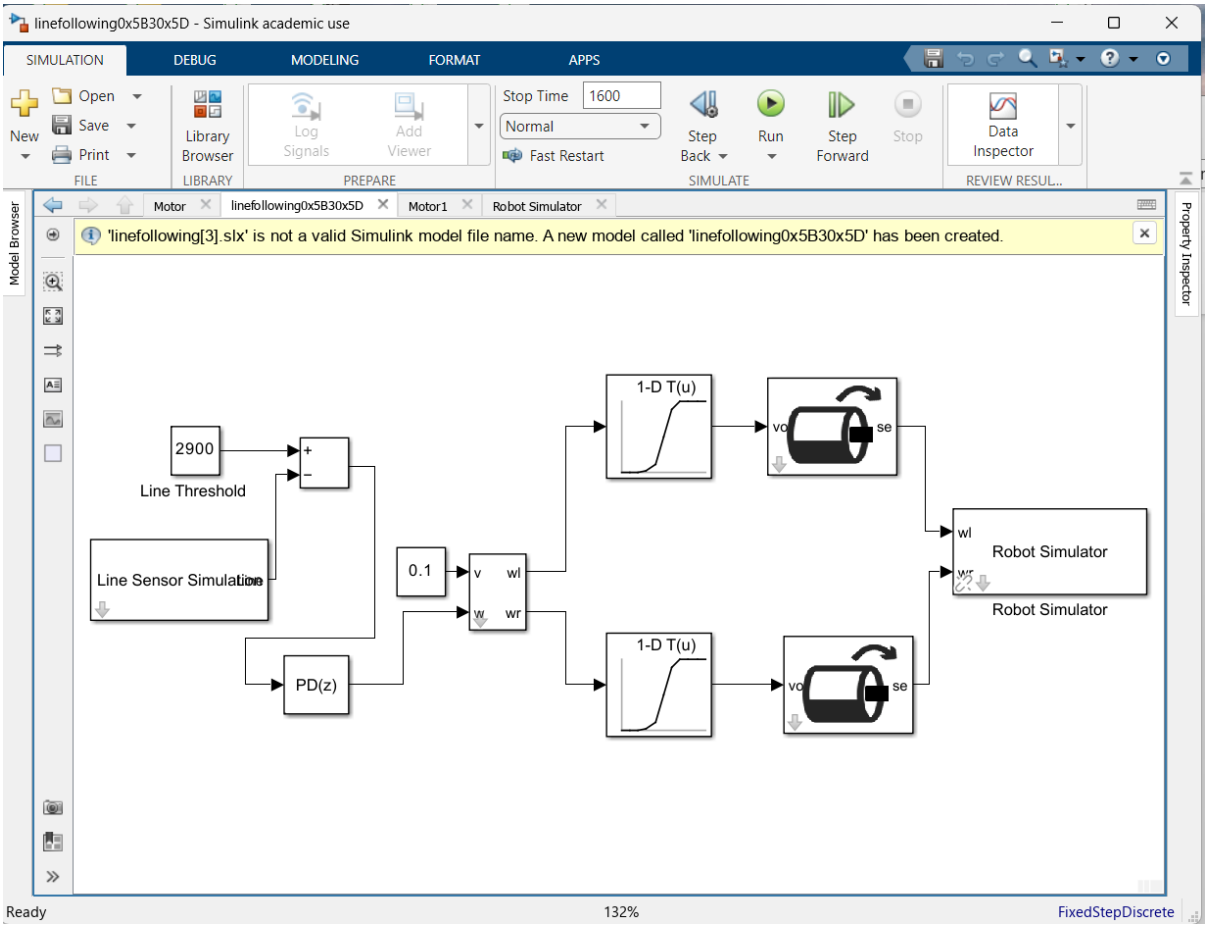


Figure 18- Circuit diagram for the simulation

In this Simulink model, the way a simple line-following robot worked using a PD (Proportional-Derivative) controller was demonstrated.

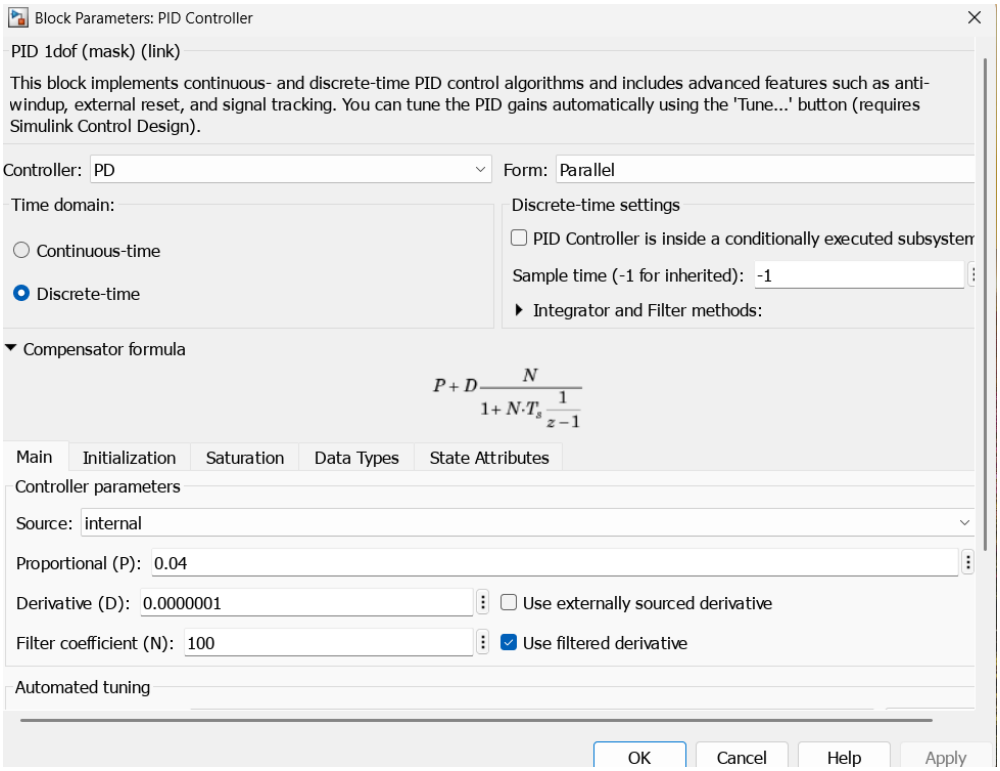


Figure 19-PID controller

A virtual line sensor was used to check whether the robot was staying on track, and its reading was compared to a set value (2900). Here we were used 8 array sensor.

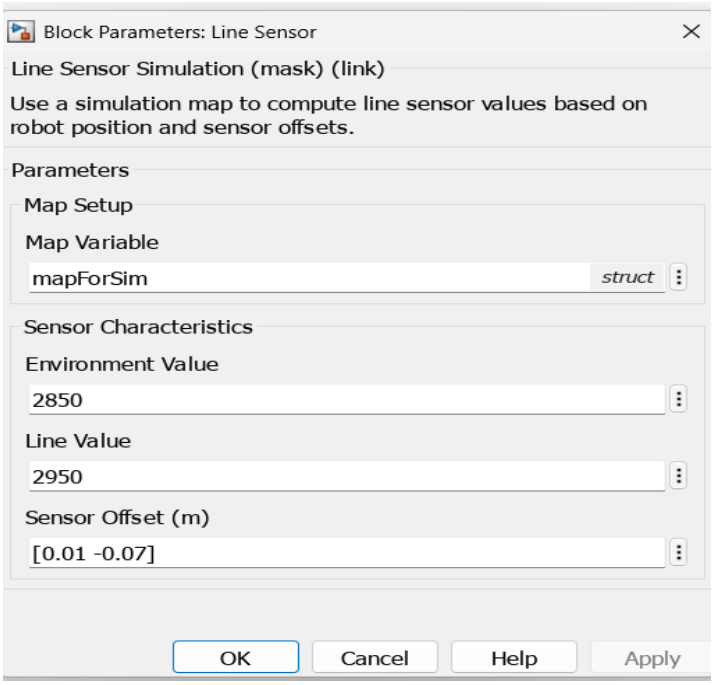


Figure 20-Block parameter for line sensor

The difference between the two was calculated to find out how far the robot had drifted from the center of the line. That error was then sent into the PD controller, where the right amount of steering correction was worked out. I used robot path which modified by map generator.

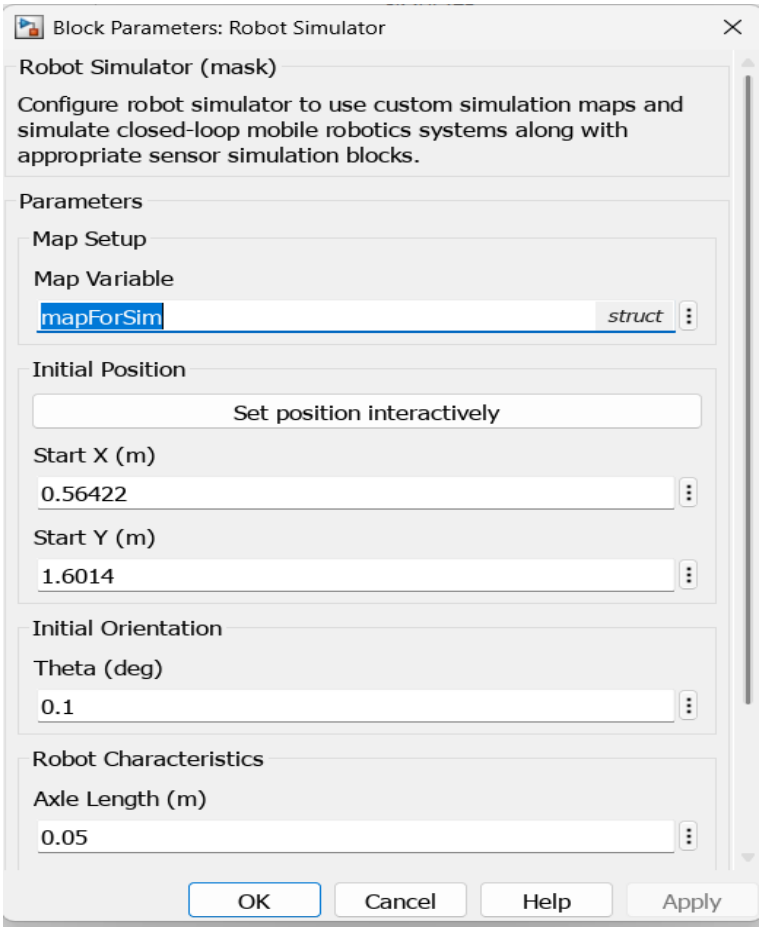


Figure 21-Block parameter for robot simulator

The robot was set to move forward at a constant speed, and the wheel speeds were automatically adjusted based on the correction-helping the robot gently steer left or right when needed. These speeds were processed through blocks that mimicked real motor behavior, and the final movement was shown in a simulated robot environment.

- Result for MATLAB simulation for line following.

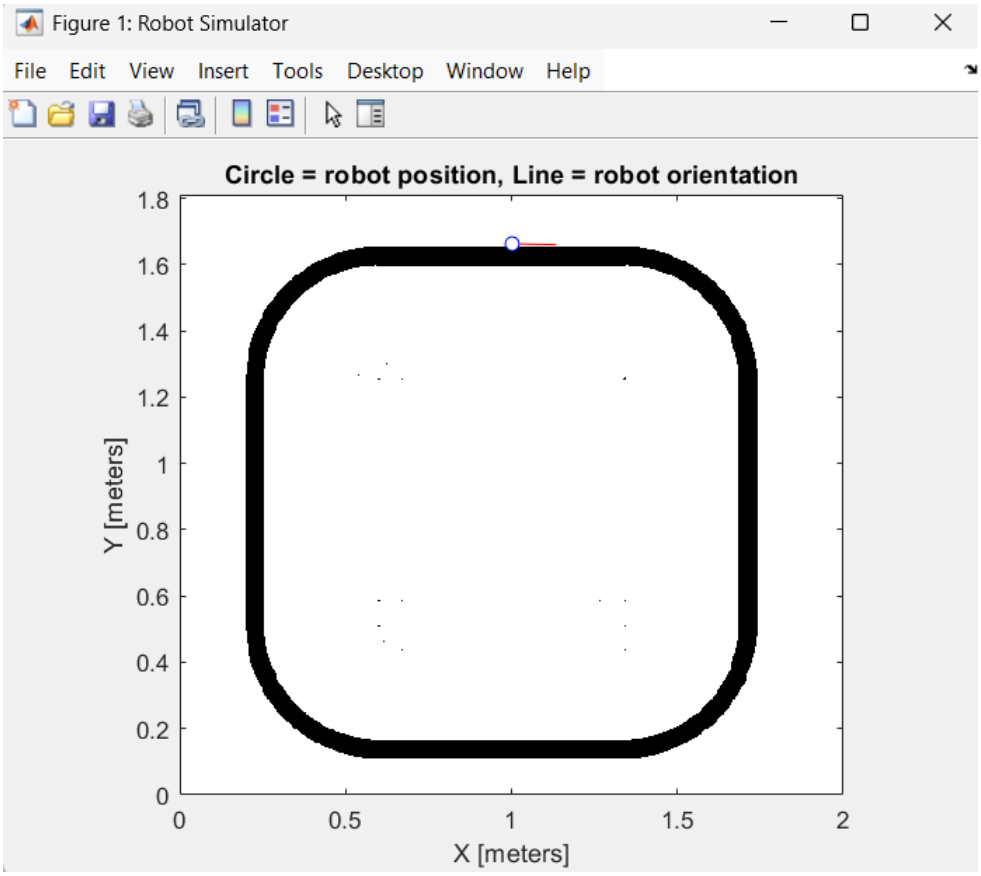


Figure 22-Result for the line following simulation

In this project, a line-following robot was successfully simulated using MATLAB. The robot was designed to follow a predefined path accurately by using sensor feedback and control logic. The simulation demonstrated that the robot could follow the line smoothly and with minimal deviation, confirming the effectiveness of the control system and algorithm. This validates the practical use of MATLAB for testing and analyzing mobile robot behavior in a virtual environment.

The success of the simulation provides a strong foundation for building the real line-following robot. It allows us to better understand the system behavior, test performance, and make improvements before implementing the design in hardware. Overall, the project achieved its goal and will now guide the development of a reliable and accurate physical robot in the next phase.

- ANSYS Simulation of Robot Arm

To check the strength and safety of the robot arm, we did a simulation using ANSYS software**.** The simulation was done without including the servo motors, but we connected one handle part to the servo motor position to complete the assembly.so we did this from two separate parts. This allowed us to properly test how the full robot arm would behave under load.

We focused on two main parts of the arm.

1. The handle part.
2. The support part.

By simulating these two parts, we were able to check stress levels, deformation, and the overall strength of the design. This helped us confirm that the robot arm could safely carry objects without bending or breaking.

- ✓ Handle

In the ANSYS simulation, we applied the material PLA (Polylactic Acid) to the robot arm model. PLA has a tensile yield strength of about 50 MPa**,** Young's modulus of 3.8GPa**,** and a Poisson's ratio of 0.36, which makes it strong and stiff for 3D-printed parts. A force was applied to the gripping face to simulate the weight of the object, and the joint of the arm was fixed to hold it in place. Applying load is 1N for full arm but load of one handle is 0.5N.

Properties of Outline Row 3: PLA

	A	B	C	D	E
1	Property	Value	Unit		
2	Material Field Variables	Table			
3	Density	1.24	g cm^-3		
4	Isotropic Elasticity				
5	Derive from	Young's Modulus an...			
6	Young's Modulus	3.8	GPa		
7	Poisson's Ratio	0.36			
8	Bulk Modulus	4.5238E+09	Pa		
9	Shear Modulus	1.3971E+09	Pa		
10	Tensile Yield Strength	50	MPa		

Figure 23-Material properties

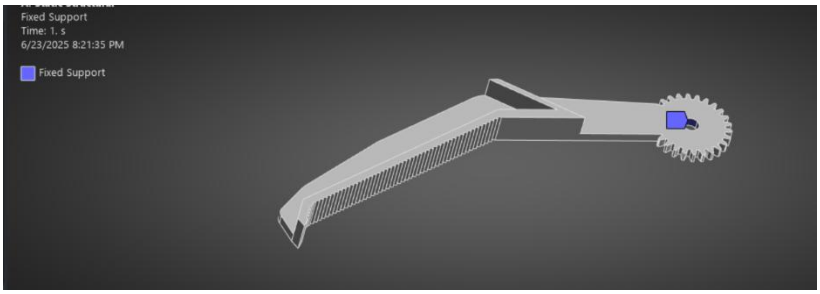


Figure 24-fixed support

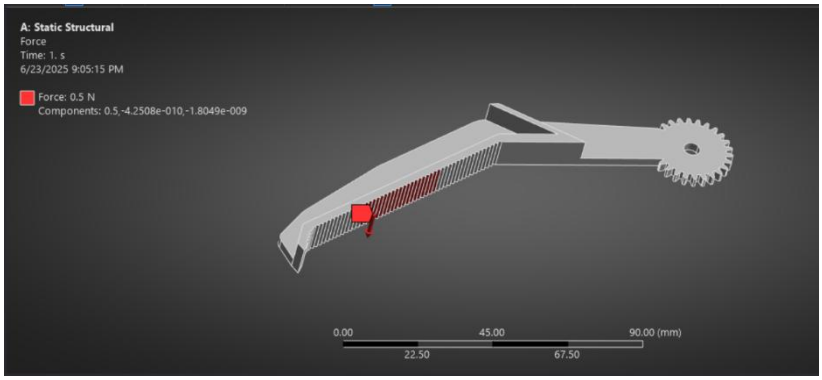


Figure 25-Applying load [0.5N]

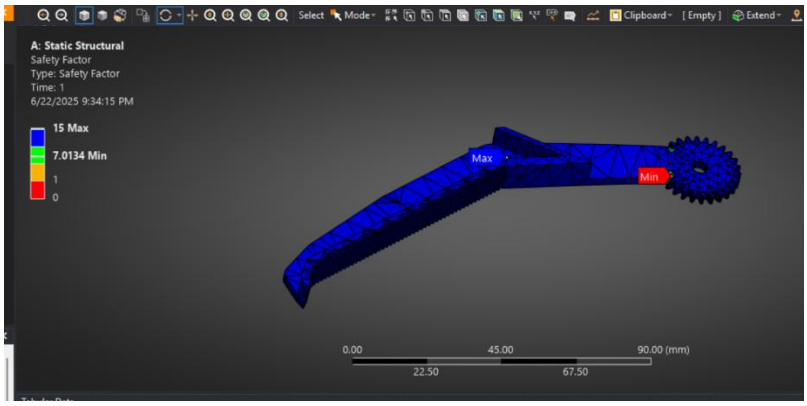


Figure 26-Safty factor

After the simulation, ANSYS gave a safety factor of 15, which means the arm is much stronger than needed. Since this is already higher than the normal safety factor of 2, we did not make any changes to reduce the strength because we had limited time. The current design is safe, strong, and ready for use.

- ✓ Support part

We also performed an ANSYS simulation for the support part of the robot arm. In this analysis, we applied the same PLA material properties as before. The load was applied as a distributed force from two load points to simulate the actual working condition. The fixed support was set at the joint where the support connects to the robot base. After running the simulation, ANSYS also gave a safety factor of 15, confirming that the support part is as strong and reliable as the handle part. Since the safety factor is high, we decided not to change the design due to limited time, and the current design is suitable for real use.

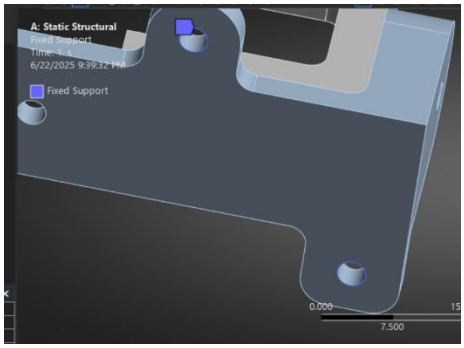


Figure 27-Fixed support

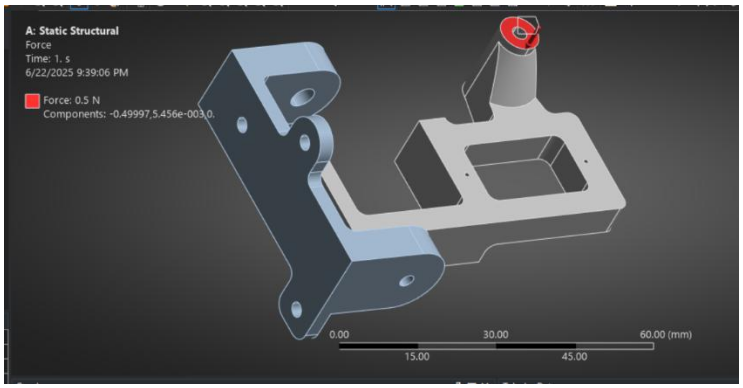


Figure 28-Force 1

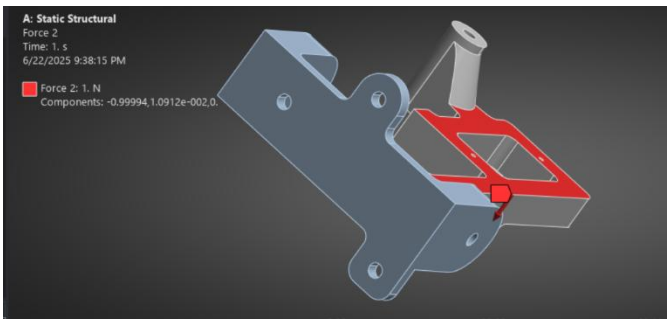


Figure 29-Force 2

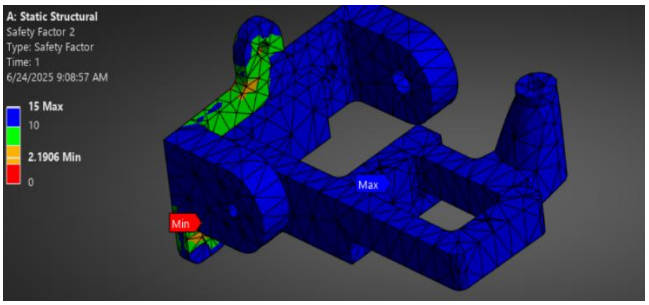


Figure 30-Safty factor

CHALLENGES ENCOUNTERED AND SOLUTIONS IMPLEMENTED.

1. Raspberry Pi board stopped working.

Challenge
At the beginning of the project, we planned to use a Raspberry Pi board as the main controller. However, the Raspberry Pi stopped working due to a technical issue. This caused a delay in the project and created a problem because we needed many input pins for sensors and motor control.

Solution
To solve this, we replaced the Raspberry Pi with an ESP32 and an Arduino Nano**.** By combining these two boards, we were able to get more input terminals for our sensors and motors. The ESP32 handled the main logic and communication, while the Arduino Nano supported additional sensor inputs. This solution allowed the robot to function properly without needing the Raspberry Pi.

2. N20 motors were not Synchronized.

Challenge
The N20 gear motors we used did not rotate at the same speed. This made the robot move slightly off the line or turn unevenly. The motors were not fully synchronized, which affected the robot's movement and accuracy.

Solution
Since we selected the N20 motors to match our limited budget, we decided to solve the issue through testing and debugging**.** We tested each motor separately, measured their speed, and adjusted the motor control values in the code. After several trials, we were able to balance the motor speeds and make the robot move more smoothly.

3. Hardware components got damaged.

Challenge
During the testing stage of the project, some hardware components stopped working. One ESP32 board, three servo motors, and one IR sensor were damaged. This affected the robot's ability to function properly and delayed our progress.

Solution
To fix the issue, we replaced the damaged parts with new ones. We were careful to test all connections and power supply settings to prevent further damage. After replacing the ESP32, servo motors, and IR sensor, the robot started working correctly again.

4. Difficulty managing too many jumper wires.

Challenge
In the early stage of the project, we used many jumper wires to connect all the components. This made the wiring look messy, and it was very hard to manage, update, or fix when something went wrong. It also increased the risk of loose connections.

Solution
To make the wiring easier and more reliable, we planned a new method using a dot board and short, neat wires**.** We also added main 5V and 3.3V power distribution lines on the board to supply power to all components properly. This new wiring setup helped to reduce clutter and made the system easier to troubleshoot and modify.

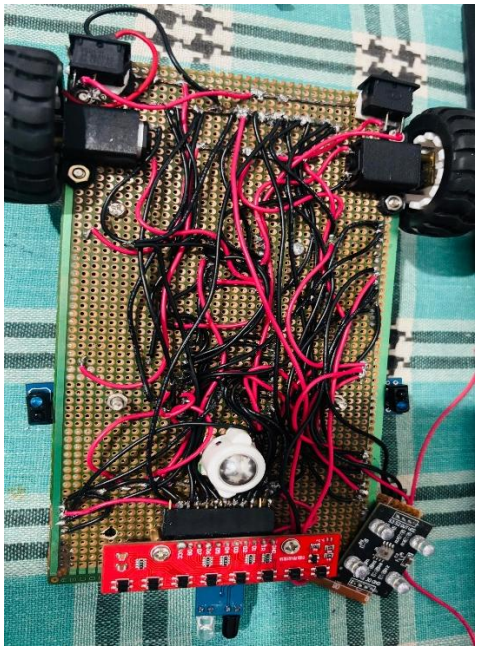


Figure 31- Wiring for robot

5. Line following not working after HMI connection.

Challenge
When we connected the HMI using Wi-Fi, the line-following function stopped working. We checked all connections and code but could not find the issue for about four days. The IR sensors were not giving correct analog values when Wi-Fi was turned on.

Solution
After researching and reading the ESP32 reference manual, we found that some analog pins do not work properly while Wi-Fi is active. To solve this, we turned off the Wi-Fi during line-following, and the sensors started working correctly again. This helped us continue both HMI and sensor functions without conflict.

6. Not enough esp32 pins for all components

Challenge
We needed many input pins for our robot, especially for the 8 pins for QTR sensor and the PID control system. But the available number of pins on the ESP32 was not enough to connect all the parts.

Solution
To solve this problem, we added an Arduino Nano and connected it to the ESP32 using serial communication. The Arduino Nano handled the QTR sensors, and the ESP32 managed the main control and communication. This setup gave us more pins and allowed the full system to work smoothly. ---[1]

7. Rear wheels lost traction after adding the arm.

Challenge
At first, the robot worked perfectly without the arm. But after attaching the arm, the rear wheels slipped and could not get enough friction. This happened because the front part of the robot became heavier than the rear, making the back wheels lose grip. We tested this by placing small soap bars on the rear part to add weight, which helped improve the grip temporarily.

Solution
To fix this permanently, we moved the robot's center of gravity toward the rear by leaning the robot arm slightly backward. This increased the weight on the back wheels and helped them get better traction. After this change, the robot moved smoothly without slipping.

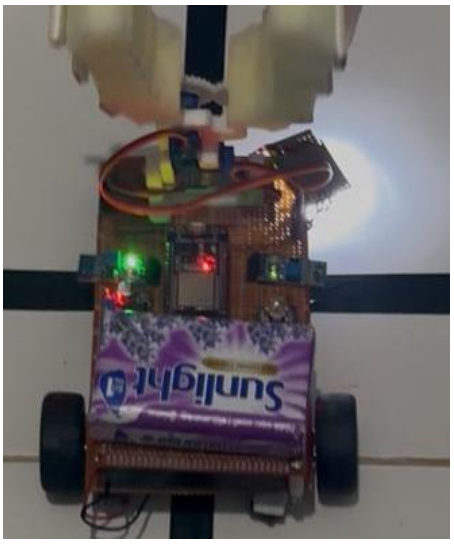


Figure 32- Moment of adding weight for back.

8. Servo motor speed was too high.

Challenge
The servo motors moved too fast, which caused the robot arm to move quickly and sometimes lose accuracy in picking or placing objects.

Solution
To slow down the servo motors, we added a delay inside the control loop. This reduced the speed by giving the motors more time between movements. After adding the delay, the arm moved smoothly and with better control.

9. Limited space for color sensor due to fixed robot length.

Challenge
In the first stage, the robot length was fixed according to the guidelines, so we could not increase it. Because of this, there was no space to place the color sensor properly. We added the sensor on the right side at an angle, but it did not work perfectly when the robot moved straight.

Solution
To fix this, we adjusted the robot's center position in the control code. This helped the robot correct its path so the color sensor could detect objects better, even though it was placed at an angle. After this change, the sensor worked more accurately during straight movement.

10. Problems initializing the TOF sensor.

Challenge
We added a TOF sensor to the robot for object detection. But after adding it, we had problems initializing the sensor. It did not start or work properly.

Solution
Because of this issue, we decided to use a different object detection sensor instead of the TOF. The new sensor worked well and helped the robot detect objects correctly.

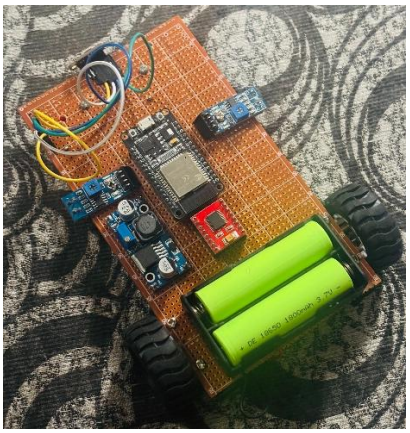


Figure 33-The robot with TOF sensor

11. Motor controller could not control motor speed.

Challenge
At the beginning, the motor controller we used could not control the speed of the motors properly. This made it hard to adjust the robot's movement speed as needed.

Solution To fix this, we replaced the old motor controller with a new one that allowed better speed control.

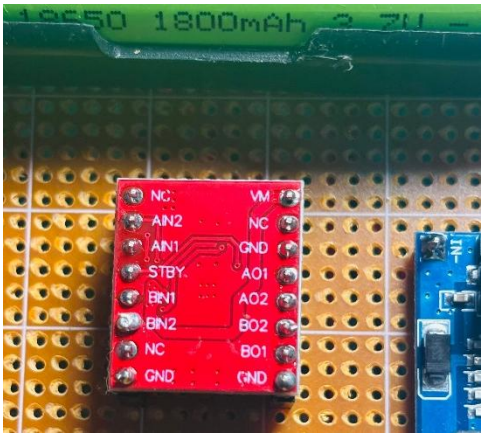


Figure 34-Previous motor controller.

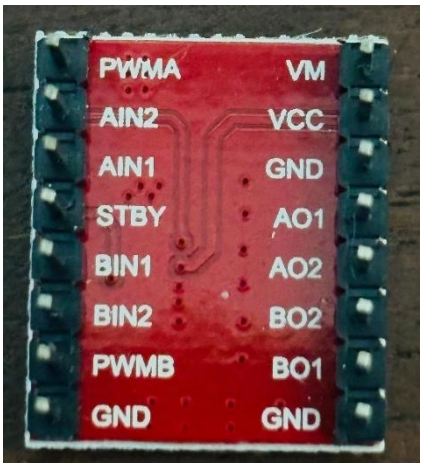


Figure 35-Current motor controller.

The reason was that the controller did not support PWM of first one, which is needed for adjusting motor speed.

✓ Drive link for all Images and Videos of project.

https://drive.google.com/drive/folders/1JGnJqJ8WAcm68tUnF2EOQDLzoXb3R-Ez

REFERENCES

[1] Espressif Systems, *ESP32 Technical Reference Manual*, version 4.4, Jun. 2020. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[2] Arduino, *Arduino Nano 3.0 Digital & Analog I/O Board – Schematic and Reference*, Rev. 2.3, 2018. [Online]. Available: https://www.arduino.cc/en/uploads/Main/ArduinoNanoManual23.pdf

[3] FMK Motorized, *PC02 Photocell Manual*, 2022. [Online]. Available: https://www.fmk-motorized.com/Upload/download/4/ManualofPC02photocell-77c909b8972f43c590fa8a03cf4c556e.pdf

[4] Raspberry Pi Press, *The Official Raspberry Pi Beginner's Guide*, 4th ed., Nov. 2020. [Online]. Available: https://magazines-attachments.raspberrypi.org/books/full_pdfs/000/000/038/original/BeginnersGuide-4thEd-Eng_v2.pdf

[5] Monolithic Power Systems, *Buck Converters (Step-Down Converter)*. [Online]. Available: https://www.monolithicpower.com/en/learning/mpscholar/power-electronics/dc-dc-converters/buck-converters.

[6] Handson Technology, *G12-N20 Geared Mini DC Motor Datasheet*, 2016. [Online]. Available: https://www.handsontec.com/dataspecs/GA12-N20.pdf

[7] Anova9347, "Line Follower Robot (with PID controller)," *Arduino Project Hub*, Aug. 2, 2020. [Online]. Available: https://projecthub.arduino.cc/anova9347/line-follower-robot-with-pid-controller-01813f

[8] Author(s) Unknown, *[Title Unknown]*, *CORE*, accessed Jun. 23, 2025. [Online]. Available: https://core.ac.uk/download/pdf/53189911.pdf

[9] W. K. Mohammed, H. D. A. Jabar, M. A. Taha, and S. A. A. Alsaidi, "Object Detection Techniques: A Review," *Wasit Journal of Computer and Mathematics Science*, vol. 2, no. 3, pp. 58–66, Sep. 2023, doi: 10.31185/wjcms.165. [Online]. Available: https://www.researchgate.net/publication/374645856_Object_Detection_Techniques_A_Review